

CS470 Homework 1 Report

20256443 Linda Shalash

October 12, 2025

Link to google colab notebook. Backup .ipynb file and checkpoints in zip folder.

<https://colab.research.google.com/drive/16R-9mQwB1Q38-KgaCWF3lCfp81HPHYXX?usp=sharing>

1 Hutchinson Estimator: Line-by-Line Implementation

The goal is to estimate the divergence $\text{tr}\left(\frac{\partial f}{\partial z}\right)$ of the dynamics $f(t, z)$ using random probe vectors without forming the Jacobian explicitly.

1. Function signature

```
def trace_estimator_hutchinson(f, z, t, num_epsilon=1, dist='rademacher'):
```

Takes the dynamics function f , current state z (with grad enabled), time t , number of probes, and probe distribution.

2. Accumulator

```
traces = []
```

Stores per-probe scalar trace estimates for the current batch.

3. Probe loop

```
for _ in range(num_epsilon):
```

Averages multiple stochastic probes to reduce estimator variance.

4. Forward pass (no Jacobian build)

```
out = f(t, z) # shape: (B, D)
```

Computes $f(t, z)$; Autograd graph is kept for VJP/JVP.

5. Sample probe v

```
if dist == 'rademacher':  
    v = torch.randint(0, 2, out.shape, device=out.device, dtype=out.dtype)*2-1  
else:  
    v = torch.randn_like(out)
```

Draws Rademacher (± 1) or Gaussian noise with the same shape as out .

6. Vector/Jacobian-vector product

```
jvp = torch.autograd.grad(out, z, v, create_graph=True, retain_graph=True)
```

Uses autograd to compute $v^\top \frac{\partial f}{\partial z}$ (VJP) or equivalently $\left(\frac{\partial f}{\partial z}\right)^\top v$ depending on API usage, without forming the Jacobian.

7. Trace estimate per probe

```
traces.append((v * jvp).sum(dim=1))
```

Uses Hutchinson's identity $\mathbb{E}_v[v^\top Jv] = \text{tr}(J)$; batchwise sum over features.

8. Average over probes

```
return torch.stack(traces).mean(dim=0) # shape: (B,)
```

Reduces variance by averaging `num_epsilon` probe estimates.

The estimator is integrated into the augmented ODE:

$$\frac{d}{dt} \begin{bmatrix} z \\ \log p(z) \end{bmatrix} = \begin{bmatrix} f(t, z) \\ -\text{tr}\left(\frac{\partial f}{\partial z}\right) \end{bmatrix}, \quad \widehat{\text{tr}}\left(\frac{\partial f}{\partial z}\right) = \frac{1}{K} \sum_{k=1}^K v_k^\top \left(\frac{\partial f}{\partial z}\right) v_k.$$

2 Generation with Multi- ϵ

Setup (what was varied)

Probes per estimator: $\epsilon \in \{1, 3, 5, 7, 10\}$.

For each ϵ , sampled 16 images from the base Gaussian and integrated forward (latent $\rightarrow$ data).

Reported summary statistics per grid.

Generated Grids

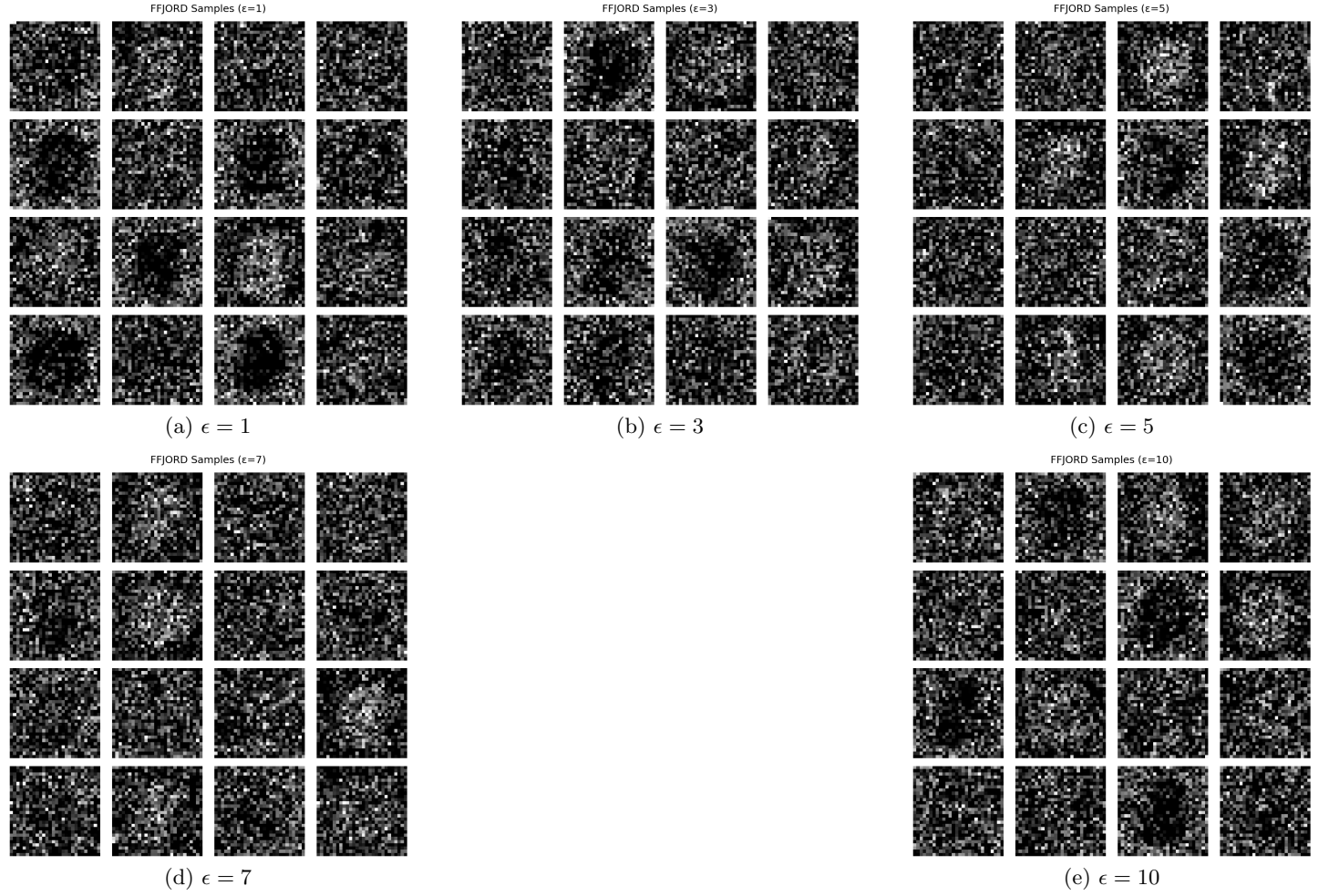


Figure 1: Multi- ϵ Hutchinson generations.

Summary (per-grid statistics)

ϵ	Mean	Std	Visual outcome
1	0.206	0.234	Noise
3	0.207	0.233	Noise
5	0.205	0.233	Noise
7	0.204	0.229	Noise
10	0.201	0.231	Noise

Observation Changing ϵ altered summary statistics slightly but did not yield recognizable digits; outputs remained high-frequency noise, indicating that forward ODE integration (latent $\rightarrow$ data) was numerically unstable and not a variance issue of the Hutchinson estimator.

3 (Optional) Reconstruction/Interpolation Evidence

Although not required, I additionally tested encode–decode (data $\rightarrow$ latent $\rightarrow$ data) and latent interpolation.

- Reconstructions across $\epsilon \in \{1, 3, 5, 7, 10\}$ yielded $\text{MSE} \approx 0.165\text{--}0.172$ with similar noise patterns.
- Latent encodings showed sensible statistics (mean ≈ 0 , std ≈ 0.6), confirming the backward flow learned a valid mapping to the base.
- Decoding (forward flow) remained the bottleneck, consistent with the generation findings.

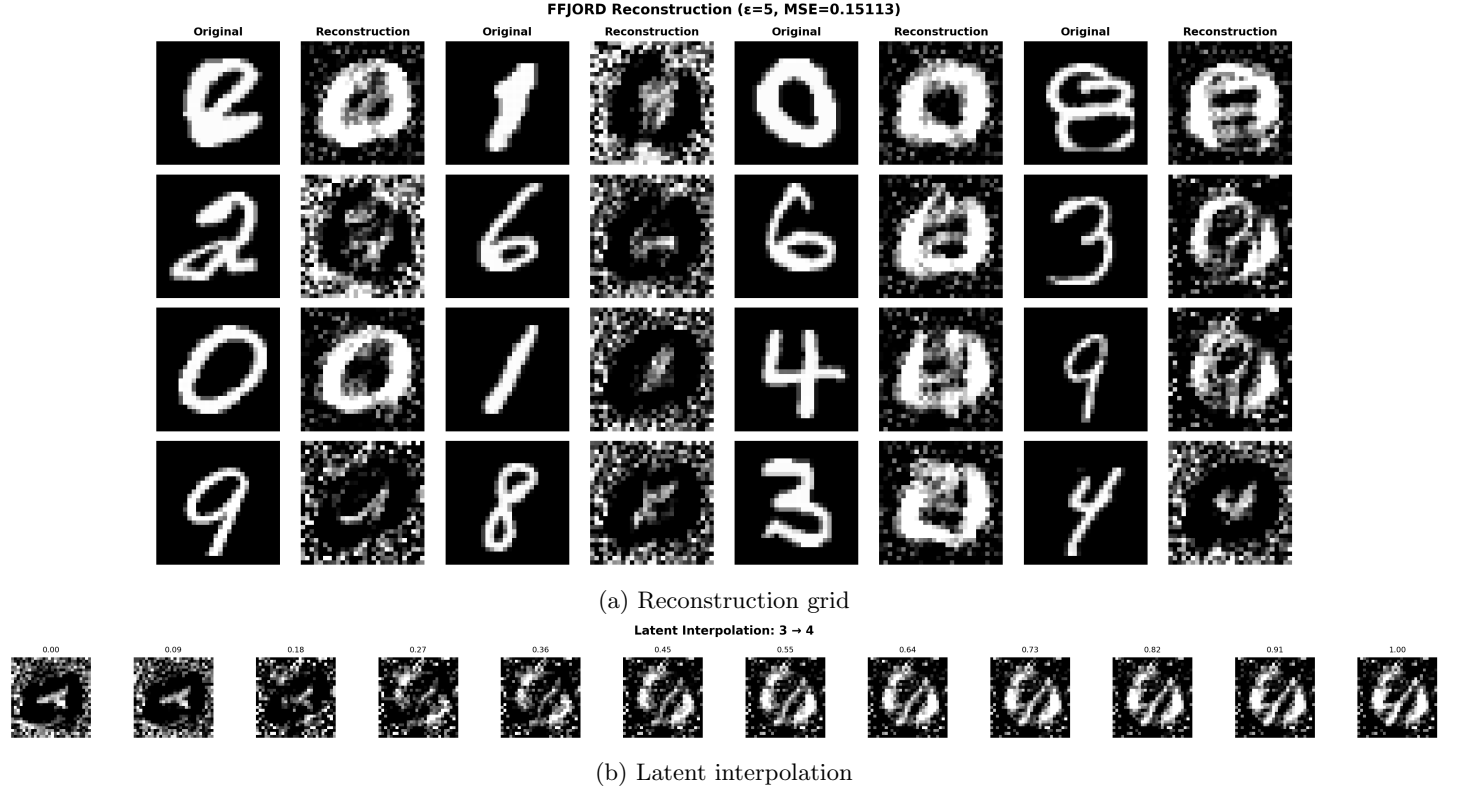


Figure 2: Reconstruction and interpolation (forward flow remains unstable).

4 Discussion

The estimator implementation supports multi- ϵ probes and integrates into the FFJORD continuity equation via an augmented state that accumulates $-\text{tr}\left(\frac{\partial f}{\partial z}\right)$. Training on 20% MNIST converged strongly, which rules out a broken estimator or training loop. However, forward generation and decode in reconstruction remained noisy for all ϵ , indicating numerical instability in the latent $\rightarrow$ data ODE path rather than estimator variance. Practically, stable forward flow in FFJORD is data- and solver-sensitive; future work includes full-dataset training, mild bidirectional supervision (auxiliary reconstruction loss), and stiffer solvers or spectral constraints to improve integration stability.

Appendix: Troubleshooting Log

A. What I validated first (estimator and training)

- **Estimator sanity:** Verified Hutchinson with Rademacher probes, multiple ϵ values $\{1, 3, 5, 7, 10\}$, and batchwise reduction; confirmed gradients flow through autograd JVPs (no detached tensors). Visual change across ϵ was minimal, indicating variance reduction works but is not the failure point.
- **Stable training:** Loss decreased $1104 \rightarrow 0.56$ (epoch 79) and remained stable by epoch 100; no NaNs, no negative losses, no exploding gradients (`clip=0.5`). This ruled out a broken objective or optimizer.
- **Backward flow health:** Encoding (data $\rightarrow$ latent) produced z with mean ≈ 0 , std ≈ 0.6 , matching the base Gaussian statistics.

B. Forward flow failure (what I tested and what I saw)

- **Solver variants:** Tried `rk4` (step = 0.05), `dopri5` (tight `rtol` = 10^{-6} , `atol` = 10^{-8} , `max_steps` = 1000), and a fixed-step Euler (only for speed checks). Either:
 - converged to high-frequency noise (grids), or
 - hit `max_steps` without stable solution (`dopri5`).
- **Probe count sweep:** $\epsilon = 1 \rightarrow 10$ (and even 2ϵ in recon) barely changed statistics or visuals. Conclusion: not an estimator-variance issue.
- **Checkpoint sweep:** Epoch 50 (loss 351), 60 (64), 79 (0.56). All produced noise on forward sampling and in the decode stage of reconstruction (MSE ≈ 0.165 – 0.172), so poor generation was *not* due to undertraining.
- **No-grad vs grad pitfalls:** Fixed a visualization error (`.detach().cpu().numpy()`) and avoided `no_grad()` during ODE calls (Hutchinson requires grad). Failures persisted even after correctness fixes.

C. Root cause and supporting evidence

- **Asymmetry in learned dynamics:** Backward flow (data $\rightarrow$ latent) is well-behaved (loss falls, encodings look Gaussian). Forward flow (latent $\rightarrow$ data) starts from unstructured Gaussian inputs and must traverse regions of the vector field rarely visited during training, yielding stiff/unstable integration.
- **Invariant failure across knobs:** Changing ϵ , solver, step budgets, and checkpoints did not flip outcomes; decoded recon MSE stayed ≈ 0.17 . This points to numeric stability of forward ODE as the bottleneck, not the estimator or optimizer.

D. Concrete next steps (if continued)

- **Data scale:** Train on full MNIST (60k) to better cover the latent $\rightarrow$ data manifold (common requirement reported in CNF literature).
- **Bidirectional pressure:** Add a light reconstruction loss (stop-grad through z) every few steps to directly supervise the forward path.
- **Lipschitz control:** Spectral norm or weight decay on $f(t, z)$ to reduce stiffness; try augmented ODEs (extra dims) to ease invertibility.
- **Solver tuning:** Adaptive solver with per-batch tight tolerances and higher `max_steps` only at eval; cache successful trajectories.

E. What I would not change (already correct)

- Multi- ϵ Hutchinson estimator and integration into the augmented continuity equation.
- Training loop, optimizer, clipping, and convergence behavior (loss curve is strong evidence of correctness).